Práctica 2

Comunicaciones en Red y Concurrencia

Angiolina Meyer

Sistemas Operativos – Universidad de Alicante

Índice

Capítulo 1....4
1.1 Enunciado del Ejercicio....4
1.1.1. Estructura de la aplicación....4
1.1.2. Resultado Esperado....5
1.2. Identificación y Justificación de Tareas....5
1.2.1. Inicialización, Validación de Parámetros y Apertura de Extremos de Red....5
1.2.2. Publicación del Servicio y Establecimiento del Canal Virtual....6
1.2.3. Delegación Concurrente sin Bloqueo del Proceso Padre....6
1.2.4. Transferencia Fragmentada de Datos y Detección de Fin de Transmisión....6
1.3 Diagrama de Secuencia....7
1.4. Análisis Detallado del Código Fuente....8
1.4.1. Funcionamiento interno del Servidor (Servidor.c)....8
1.4.2. Funcionamiento interno del Cliente (Cliente.c)....11
Capítulo 2....14
2.1. Enunciado del Ejercicio....14
2.1.1. Estructura de la aplicación....15
2.1.2. Resultado Esperado....15
2.2. Identificación y Justificación de Tareas....18
2.2.1. Inicialización del Buffer y de los Mecanismos de Sincronización....18
2.2.2. Trabajo Individual Fuera de la Sección Crítica....18
2.2.3. Acceso Exclusivo y Gestión Circular del Buffer....18
2.2.4. Coordinación entre Procesos y Orden de Bloqueo....18
2.3. Diagrama de Secuencia....19
2.4. Análisis Detallado del Código Fuente....20

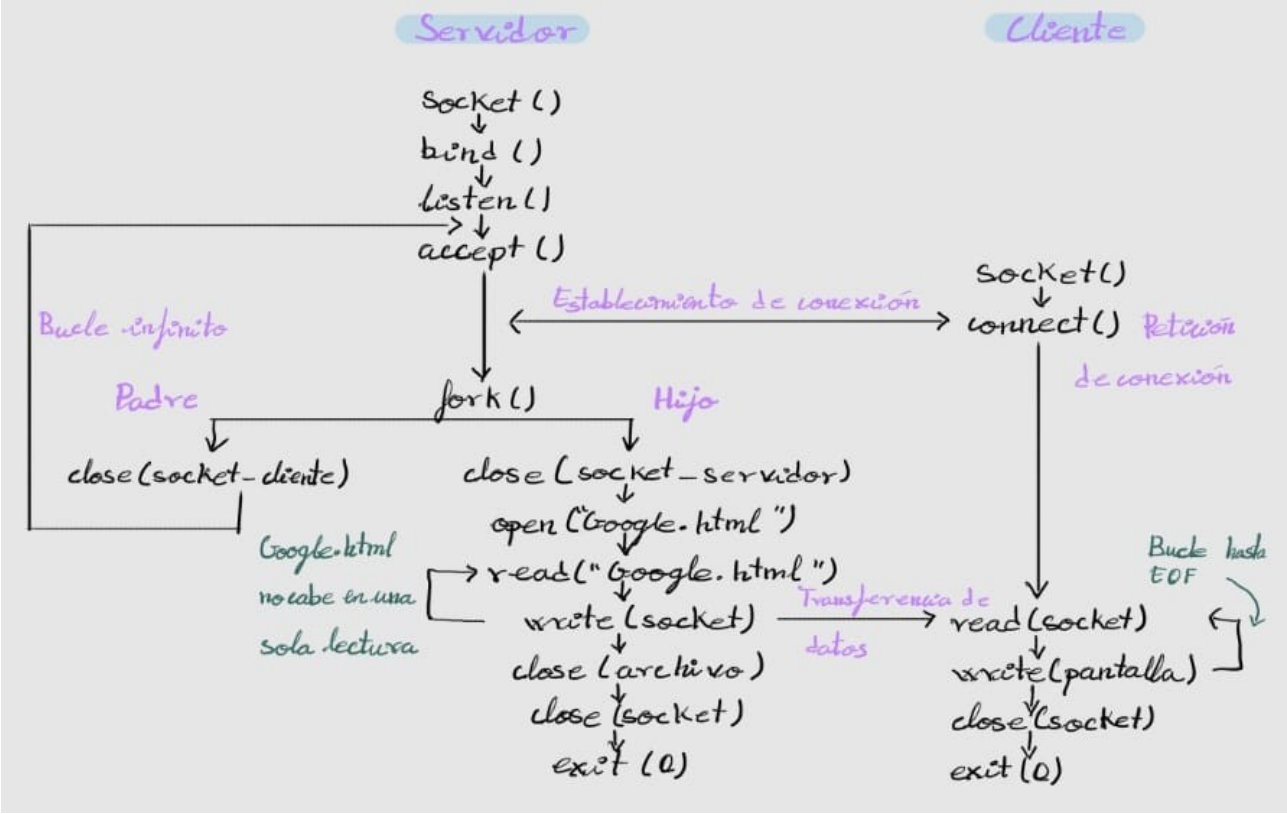
Capítulo 1

Ejercicio 1 – Comunicaciones en red

1.1 Enunciado del Ejercicio

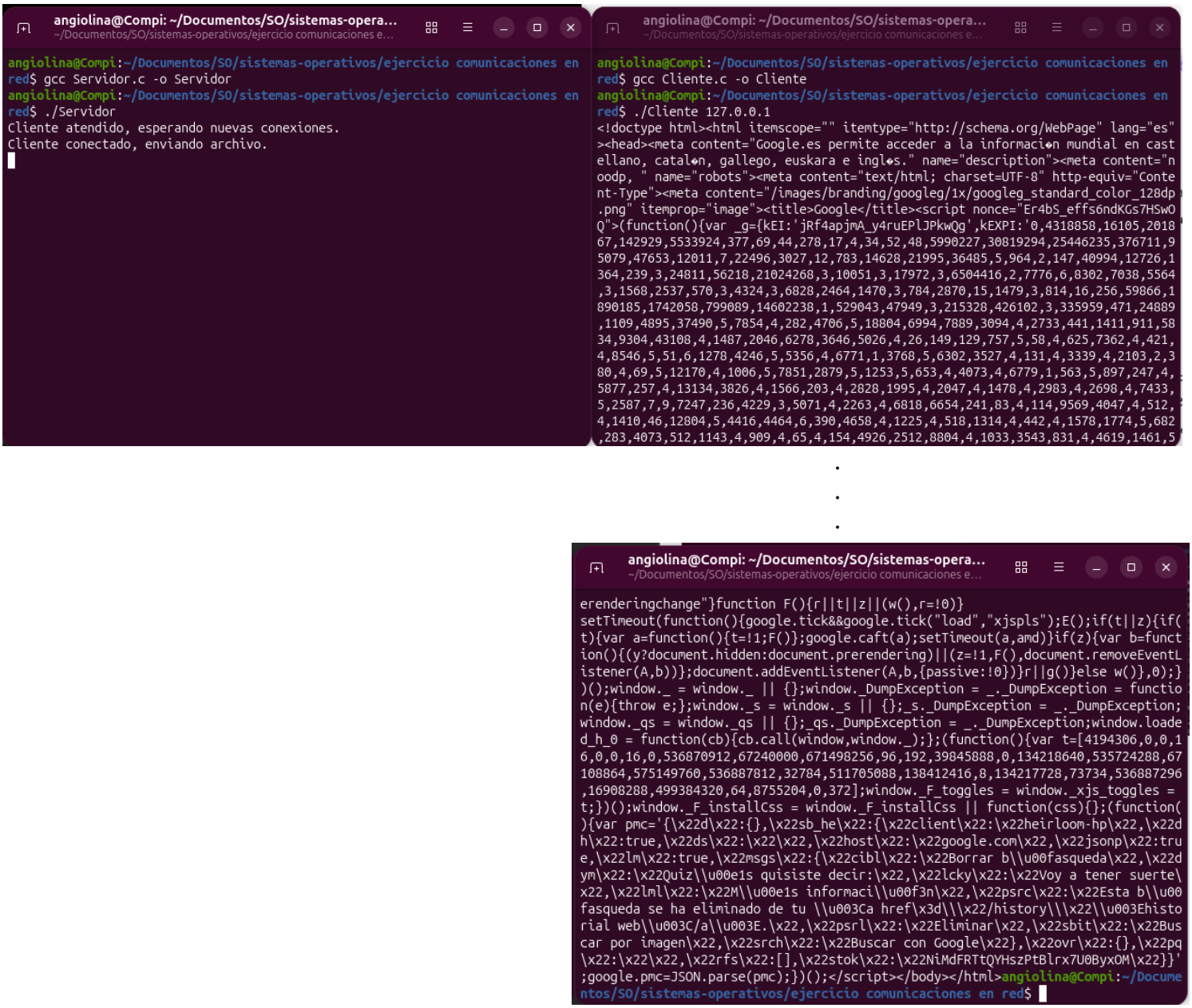
Implementar una arquitectura Cliente-Servidor concurrente en C utilizando sockets orientados a conexión bajo el protocolo IPv4. El servidor permanecerá a la escucha en el puerto 9999 aceptando peticiones entrantes de forma indefinida. Por cada cliente que se conecte, el servidor creará un proceso hijo mediante fork() encargado de leer el contenido del archivo local Google.html y enviarlo a través del socket, mientras el proceso padre regresa inmediatamente a escuchar nuevas conexiones sin utilizar wait(NULL) ni captura de señales. Por su parte, el cliente recibirá la dirección IP del servidor como argumento por línea de comandos, establecerá la conexión y volcará en su salida estándar todo el flujo de datos recibido hasta detectar el fin de la transmisión.

1.1.1. Estructura de la aplicación



1.1.2. Resultado Esperado

A la izquierda está la terminal del Servidor y a la derecha del Cliente



1.2. Identificación y Justificación de Tareas

La arquitectura de la aplicación distribuida se deriva directamente de los requisitos del enunciado, traduciendo cada restricción funcional y de concurrencia en una decisión de diseño sobre la interfaz de Sockets POSIX y la gestión de procesos en Linux. En lugar de abordar el cliente y el servidor como entidades aisladas, las tareas del sistema se estructuran de forma global siguiendo las cuatro fases del ciclo de vida de la comunicación:

1.2.1. Inicialización, Validación de Parámetros y Apertura de Extremos de Red

El requisito de que el cliente reciba la dirección destino por consola mientras el servidor opera en un puerto fijo exige una fase previa de configuración antes de iniciar cualquier intercambio de

datos. En el cliente, se valida el número de argumentos y se traduce la cadena IP a formato binario de red mediante inet_addr(), abortando de forma temprana si la dirección es inválida para no reservar recursos innecesarios del núcleo. Paralelamente, ambos procesos solicitan al sistema operativo un punto de comunicación extremo a extremo mediante socket(AF_INET, SOCK_STREAM, 0), eligiendo el protocolo orientado a conexión TCP sobre IPv4 para garantizar la entrega fiable y ordenada del código fuente HTML, y convirtiendo el número de puerto al estándar de red *Big-Endian* mediante htons().

1.2.2. Publicación del Servicio y Establecimiento del Canal Virtual

Para cumplir con la condición de que el servidor permanezca preparado en todo momento para aceptar conexiones, este asocia su descriptor a todas las interfaces locales disponibles en el puerto 9999 mediante bind() y lo transforma en un socket pasivo de escucha con listen(), habilitando una cola de espera para peticiones simultáneas. A partir de ese instante, la llamada a accept() suspende al servidor hasta que el cliente inicia el saludo de tres vías (Synchronize, Synchronize-Acknowledge, Acknowledge) mediante connect(). Al completarse este enlace, el núcleo crea en el servidor un socket activo dedicado exclusivamente a esa sesión, manteniendo intacto el socket pasivo original para futuros clientes.

1.2.3. Delegación Concurrente sin Bloqueo del Proceso Padre

El requisito explícito de que tras una petición de conexión el servidor cree un hijo encargado de la transferencia, quedando el padre libre para escuchar en todo momento, impone un modelo de concurrencia real mediante multiprocesamiento. Nada más retornar accept(), el servidor invoca fork() para bifurcar la ejecución:

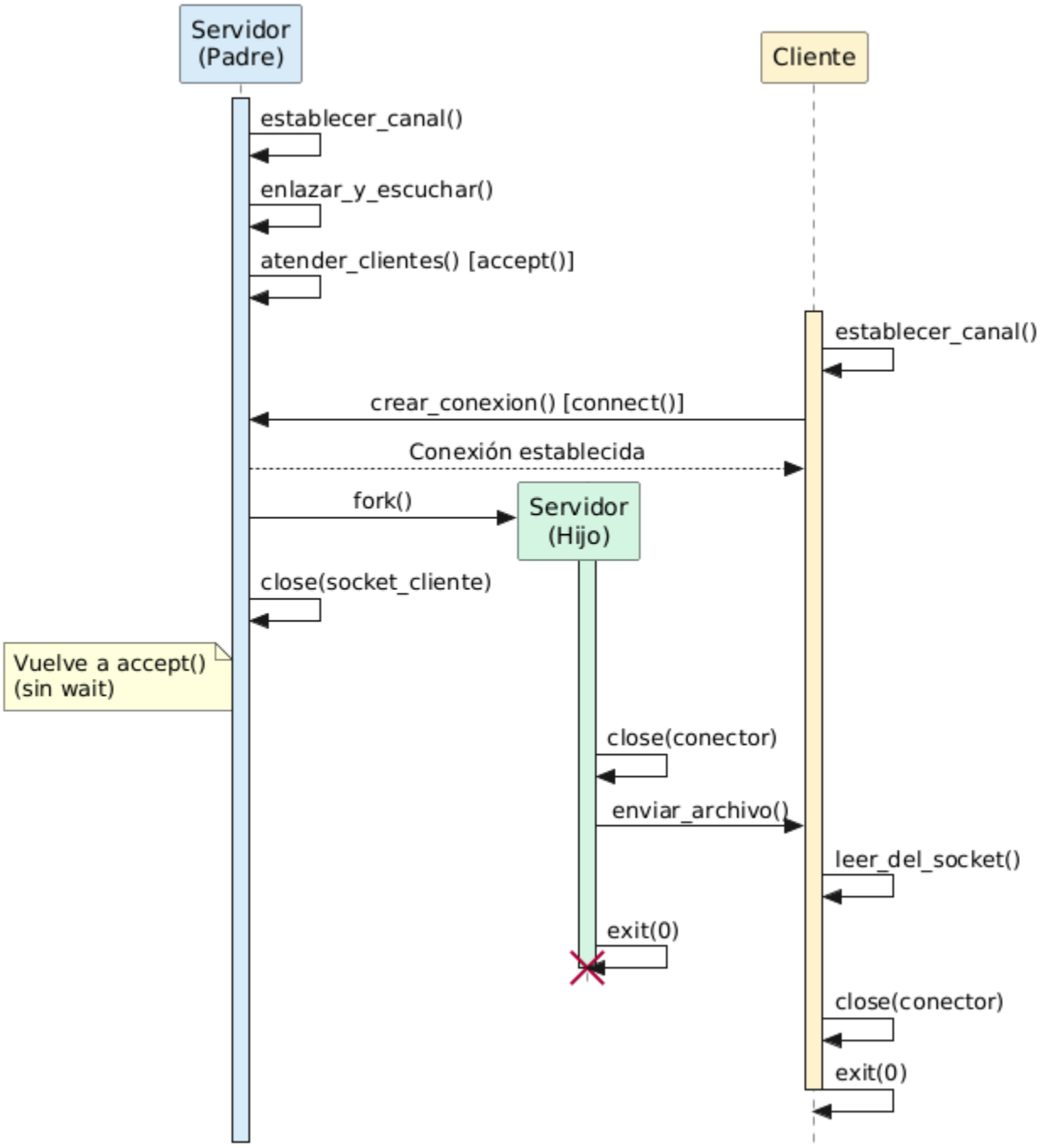
En el proceso hijo: Se cierra inmediatamente la copia heredada del socket pasivo de escucha (ya que no aceptará nuevos clientes) y se asume toda la carga de E/S de la transferencia, finalizando explícitamente con exit(0) para no entrar en el bucle infinito del padre.

En el proceso padre: Se cierra su copia del socket activo del cliente (ya que tras el fork() ambos procesos comparten el descriptor; si el padre no cerrara su referencia, la conexión permanecería abierta en el núcleo incluso cuando el hijo terminara, dejando al cliente bloqueado esperando datos) y regresa de inmediato a accept(). De forma deliberada, el padre no invoca wait(NULL), ya que quedar bloqueado esperando la terminación del hijo anularía la concurrencia y convertiría el servidor en secuencial.

1.2.4. Transferencia Fragmentada de Datos y Detección de Fin de Transmisión

Dado que el fichero Google.html supera la capacidad de una única lectura de red, la transmisión entre el proceso hijo del servidor y el cliente se resuelve mediante un flujo iterativo por bloques de 4096 bytes. El hijo abre el archivo local en modo solo lectura (O_RDONLY) y encadena llamadas read() sobre el disco con write() sobre el socket activo. Simultáneamente, el cliente ejecuta un bucle de recepción read() sobre su socket volcando cada fragmento recibido directamente hacia su salida estándar (stdout, descriptor 1). Cuando el hijo completa el envío y cierra su descriptor del socket, el protocolo TCP envía un segmento FIN que hace que el siguiente read() del cliente devuelva 0 (EOF), permitiéndole cerrar su canal y finalizar de forma ordenada.

1.3 Diagrama de Secuencia



1.4. Análisis Detallado del Código Fuente

1.4.1. Funcionamiento interno del Servidor (Servidor.c)

```c
#include<sys/socket.h>
#include<netinet/in.h>
#include<sys/types.h>
#include<arpa/inet.h>

#include<stdio.h>
#include<stdlib.h>
#include<unistd.h>
#include<fcntl.h>

const int PUERTO = 9999;

int establecer_canal()
{
    int conector = socket(AF_INET, SOCK_STREAM, 0);

    if(conector < 0)
    {
        perror("Error al crear el socket");
        exit(1);
    }

    return conector;
}
```

Creación del socket (establecer_canal): Para poder comunicarse por red, lo primero que hace el programa es pedirle al sistema operativo un punto de conexión mediante socket(AF_INET, SOCK_STREAM, 0). Se usa IPv4 (AF_INET) y el protocolo TCP (SOCK_STREAM) en lugar de UDP porque aquí no podemos permitirnos que se pierda ni se desordene ningún trozo de Google.html. Al elegir TCP, es el propio sistema operativo el que se encarga de asegurar que todos los bytes lleguen íntegros y en el orden correcto.

```c
void enlazar_y_escuchar(int conector, struct sockaddr_in *dir_servidor)
{
    if(bind(conector, (struct sockaddr *)dir_servidor, sizeof(*dir_servidor)) < 0)
    {
        perror("Error en el bind");
        exit(1);
    }

    if(listen(conector, 5) < 0)
    {
        perror("Error en el listen");
        exit(1);
    }
}
```

Asociación a la dirección y puesta en escucha (enlazar_y_escuchar): Un socket recién creado todavía no tiene una dirección asociada para que los clientes lo encuentren. Con bind() vinculamos nuestro socket a la estructura dir_servidor (donde previamente en el main() se le ha asignado el puerto 9999 y INADDR_ANY). Justo después, listen(conector, 5) transforma este socket en un

socket pasivo de escucha: a partir de este momento ya no sirve para enviar datos, sino únicamente para recibir peticiones de conexión y mantener en cola hasta 5 clientes pendientes mientras se van atendiendo.

```c
void enviar_archivo(int socket_cliente)
{
    int archivo = open("Google.html", O_RDONLY);
    if(archivo < 0)
    {
        perror("Error al abrir el archivo");
        close(socket_cliente);
        exit(1);
    }

    char buffer[4096];
    ssize_t leidos;

    while((leidos = read(archivo, buffer, sizeof(buffer))) > 0)
    {
        if(write(socket_cliente, buffer, leidos) < 0)
        {
            perror("Error al escribir en el socket");
            close(archivo);
            close(socket_cliente);
            exit(1);
        }
    }

    if(leidos < 0)
    {
        perror("Error al leer el archivo");
        close(archivo);
        close(socket_cliente);
        exit(1);
    }

    close(archivo);
    close(socket_cliente);
}
```

Lectura y envío por bloques (enviar_archivo): Como Google.html pesa bastante más de lo que cabe en una sola lectura y tampoco tiene sentido cargarlo entero de golpe en memoria, el servidor actúa como un puente: abre el archivo en modo solo lectura (O_RDONLY) y va leyendo del disco poco a poco (en bloques de 4096 bytes) para escribir cada trozo directamente en el socket_cliente. Cuando termina de leer el fichero completo, cierra tanto el archivo como el socket. Cerrar el socket en este punto es clave, ya que le indica al protocolo TCP que mande una señal de fin (FIN) para que el cliente sepa que ya no quedan más datos por recibir.

```c
void atender_clientes(int conector)
{
    while(true)
    {
        int socket_cliente = accept(conector, NULL, NULL);
        if(socket_cliente < 0)
        {
            perror("Error al aceptar la conexión");
            continue;
        }

        pid_t pid = fork();

        if(pid < 0)
        {
            perror("Error en el fork");
            close(socket_cliente);
        }
        //el hijo cierra el conector y envía el archivo al cliente
        else if(pid == 0)
        {
            close(conector);
            printf("Cliente conectado, enviando archivo.\n");
            enviar_archivo(socket_cliente);
            exit(0);
        }
        //sólo el servidor ejecuta esto
        else
        {
            close(socket_cliente);
            printf("Cliente atendido, esperando nuevas conexiones.\n");
        }
    }
}

int main()
{
    struct sockaddr_in dir_servidor = {0};

    dir_servidor.sin_family = AF_INET;//se usará el protocolo de red IPv4
    dir_servidor.sin_port = htons(PUERTO);//se asigna el puerto y se transforma al
    dir_servidor.sin_addr.s_addr = INADDR_ANY; // acepta conexiones desde cualquie

    int conector = establecer_canal();
    enlazar_y_escuchar(conector, &dir_servidor);
    atender_clientes(conector);

    close(conector);
    return 0;
}
```

Configuración inicial y atención concurrente (atender_clientes y main): En el main() se prepara la estructura dir_servidor indicando el protocolo IPv4, aceptando peticiones desde cualquier interfaz de red del equipo (INADDR_ANY) y adaptando el puerto 9999 al orden de bytes de red con htons(), para después encadenar la creación, escucha y atención del servidor.

Dentro de atender_clientes(), la función accept() deja al servidor en pausa hasta que llega un cliente. Cuando se establece la conexión, accept() devuelve un socket nuevo y exclusivo (socket_cliente)

para hablar con él, dejando libre el conector original para seguir escuchando. Para que el envío del archivo no bloquee la llegada de otros usuarios, el servidor se duplica mediante fork():

- Qué hace el hijo: Cierra su copia del socket de escucha (conector, porque él no va a aceptar nuevos clientes), llama a enviar_archivo() y termina con exit(0) para no quedarse atrapado en el bucle infinito del padre.
- Qué hace el padre: Cierra inmediatamente su copia del socket_cliente y vuelve arriba a accept() sin quedarse bloqueado en un wait(). Este cierre cruzado es obligatorio: al hacer fork(), tanto el padre como el hijo comparten el mismo socket abierto. Si el padre no cerrara su copia, el sistema operativo pensaría que la conexión sigue activa aunque el hijo terminase, y el cliente se quedaría esperando eternamente el final de la transmisión.

1.4.2. Funcionamiento interno del Cliente (Cliente.c)

```c
#include<sys/socket.h>
#include<netinet/in.h>
#include<sys/types.h>
#include<arpa/inet.h>

#include<stdio.h>
#include<stdlib.h>
#include<unistd.h>

const int PUERTO = 9999;


int establecer_canal()
{
    int conector = socket(AF_INET, SOCK_STREAM, 0);

    if(conector < 0)
    {
        perror("Error al crear el socket");
        exit(1);
    }

    return conector;
}
```

Creación del socket del cliente (establecer_canal): Al igual que en el servidor, el primer paso es pedirle al sistema operativo un extremo de comunicación IPv4 orientado a flujo TCP (socket(AF_INET, SOCK_STREAM, 0)). De esta forma, ambos programas hablan exactamente el mismo protocolo y el núcleo se encarga de reensamblar en orden todos los paquetes que lleguen desde el servidor.

```c
void leer_del_socket(int conector)
{
    char buffer[4096];
    ssize_t salida;

    while((salida = read(conector, buffer, sizeof(buffer))) > 0)//ha leído bytes
    {
        write(1, buffer, salida);
    }
    if(salida < 0)
    {
        perror("Error al leer del socket");
        exit(1);
    }

    close(conector);
    return;
}
```

Recepción por bloques y volcado por pantalla (leer_del_socket): Como el cliente no sabe de antemano cuánto ocupa el archivo que le van a enviar, se limita a ir vaciando el socket conforme van llegando los datos en bloques de hasta 4096 bytes. Cada trozo que entra por la red se manda directamente al descriptor 1 (la salida estándar del terminal) con write(). El proceso sabe que el fichero ha terminado cuando el servidor cierra su conexión: en ese momento, read() devuelve 0 (EOF), el cliente cierra su propio descriptor con close(conector) para liberar recursos y sale de la función.



```c
void crear_conexion(int conector, struct sockaddr_in *dir_servidor)
{
    if(connect(conector, (struct sockaddr *)dir_servidor, sizeof(*dir_servidor)) < 0)
    {
        perror("Error al conectar con el servidor");
        exit(1);
    }

    leer_del_socket(conector);

    return;
}

int main(int argc, char *argv[])
{
    if(argc != 2)
    {
        printf("Debes ejecutar de la forma: Cliente 'IP del servidor'.\n");
        exit(1);
    }

    //se crea el struct que contendrá la infmración del servidor que se quiere conectar
    struct sockaddr_in dir_servidor = {0};

    dir_servidor.sin_family = AF_INET;//se usará el protocolo de red IPv4
    dir_servidor.sin_port = htons(PUERTO);//se asigna el puerto y se transforma al orden de bytes es
    dir_servidor.sin_addr.s_addr = inet_addr(argv[1]);//se traduce la IP introducida por el susario

    if(dir_servidor.sin_addr.s_addr == INADDR_NONE)
    {
        printf("La IP introducida no es válida.\n");
        exit(1);
    }

    crear_conexion(establecer_canal(), &dir_servidor);
}
```

Validación de argumentos y conexión activa (crear_conexion y main): Antes de abrir ningún recurso de red, el main() comprueba que se haya pasado exactamente una dirección IP por consola (argc == 2) y la traduce de texto a formato binario con inet_addr(), cortando la ejecución de inmediato si la IP no es válida (INADDR_NONE). A diferencia del servidor, aquí no hace falta llamar a bind(), ya que el sistema operativo asigna al cliente un puerto libre automático en cuanto ejecuta connect() dentro de crear_conexion(). Una vez completado el saludo con el servidor en el puerto 9999, crear_conexion() pasa el testigo directamente a leer_del_socket() para recibir el contenido de Google.html.

Capítulo 2

Ejercicio 2 – Concurrencia

2.1. Enunciado del Ejercicio

El propósito de este segundo ejercicio es diseñar e implementar en el entorno de simulación jBACI una solución mediante semáforos para el problema del Productor-Consumidor con buffer limitado visto en teoría.

En el modelo teórico de partida, dos procesos concurrentes comparten una zona de memoria acotada que actúa como intermediaria: el productor genera datos y los deposita siempre que exista espacio libre, mientras que el consumidor los retira para procesarlos siempre que haya algún elemento disponible.

Para garantizar que ambos procesos colaboren sin interferencias, la solución de teoría emplea tres semáforos (e, s y n) que cubren las dos necesidades fundamentales de la concurrencia:

Exclusión mutua (s). Asegura que únicamente un proceso pueda acceder al buffer compartido en cada instante, evitando condiciones de carrera sobre los datos o los índices.

Sincronización condicional (e y n). Regula el ritmo entre ambos extremos sin desperdiciar ciclos de CPU. El semáforo e contabiliza los espacios vacíos disponibles (bloqueando al productor si el buffer se llena) y el semáforo n contabiliza los elementos ocupados (bloqueando al consumidor si el buffer queda vacío).

El pseudocódigo visto en las diapositivas de teoría es el siguiente:

```
#define tamaño_buffer N
TSemáforo e, s, n;

void productor()
{
  while (true)
  {
    producir();
    P(e);
    P(s);
    añadir_buffer();
    V(s);
    V(n);
  }
}

void consumidor()
{
  while (true)
  {
    P(n);
    P(s);
    coger_ buffer();
    V(s);
    V(e);
    consumir();
  }
}

void main()
{
  inicializar(s, 1);
  inicializar(n, 0);
  inicializar(e, tamaño_buffer);
  cobegin
    productor();
    consumidor();
  coend;
}
```

Partiendo de esta base teórica, el enunciado de la práctica establece dos requisitos clave de evaluación:

Implementar correctamente la sincronización y la exclusión mutua mediante semáforos en jBACI, traduciendo las operaciones teóricas P() y V() a las primitivas wait() y signal() del entorno y acotando la simulación a un número finito de iteraciones.

Incorporar una interfaz gráfica en jBACI que represente visualmente el estado del buffer y de ambos procesos en tiempo real para facilitar la comprensión de la ejecución concurrente.

2.1.1. Estructura de la aplicación

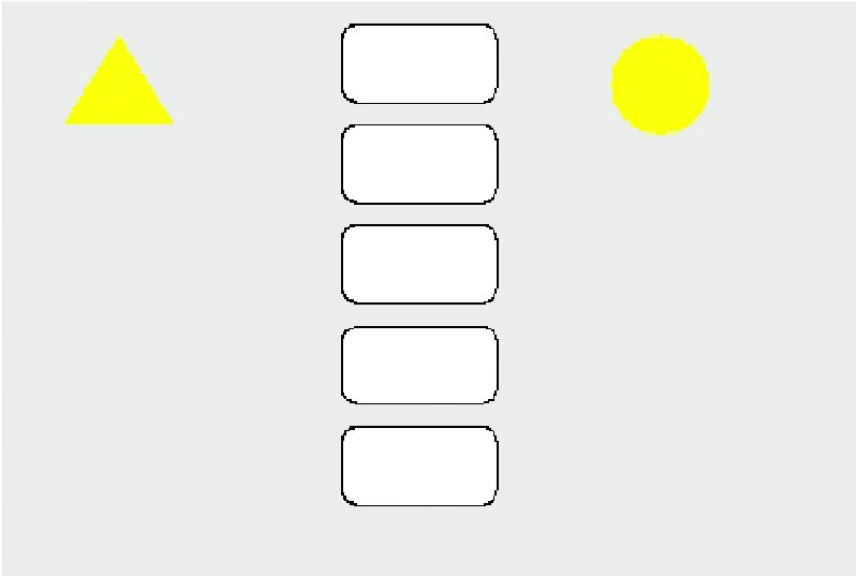
Como en el enunciado no se muestra ninguna estructura a seguir, en este apartado no pondré ninguna imagen. En el apartado 2.3. Diagrama de Secuencia sí lo haré. Aparecerá el diagrama base de mi ejercicio.

2.1.2. Resultado Esperado

A continuación se muestra la evolución visual de la simulación en la ventana gráfica de jBACI a lo largo de los distintos estados clave de la ejecución concurrente:

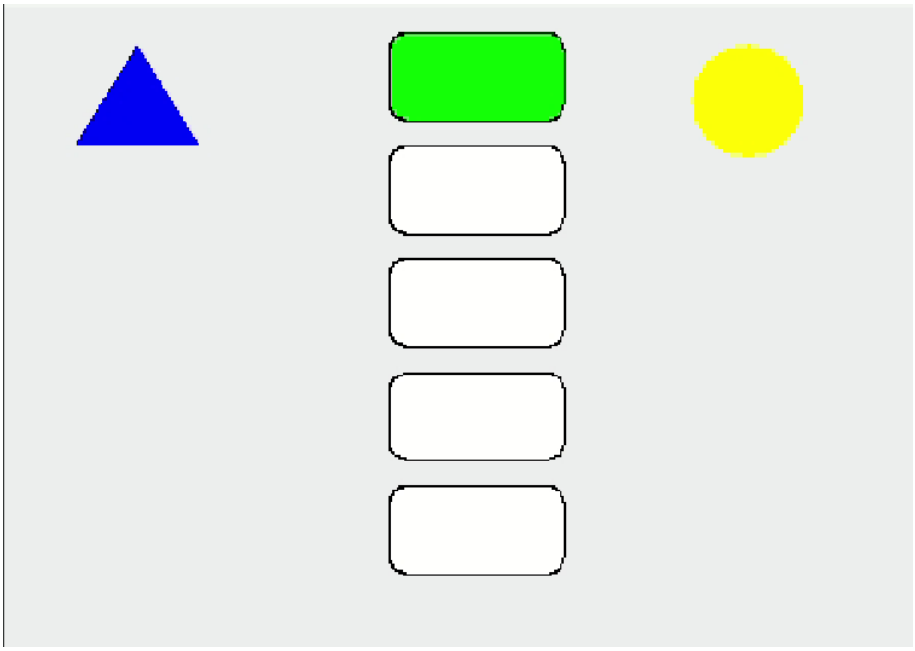
Estado inicial del sistema:

Las 5 celdas del buffer circular arrancan vacías (en blanco) y tanto el proceso productor (triángulo a la izquierda) como el consumidor (círculo a la derecha) se encuentran fuera de la sección crítica (en amarillo) antes de iniciar el intercambio de datos.



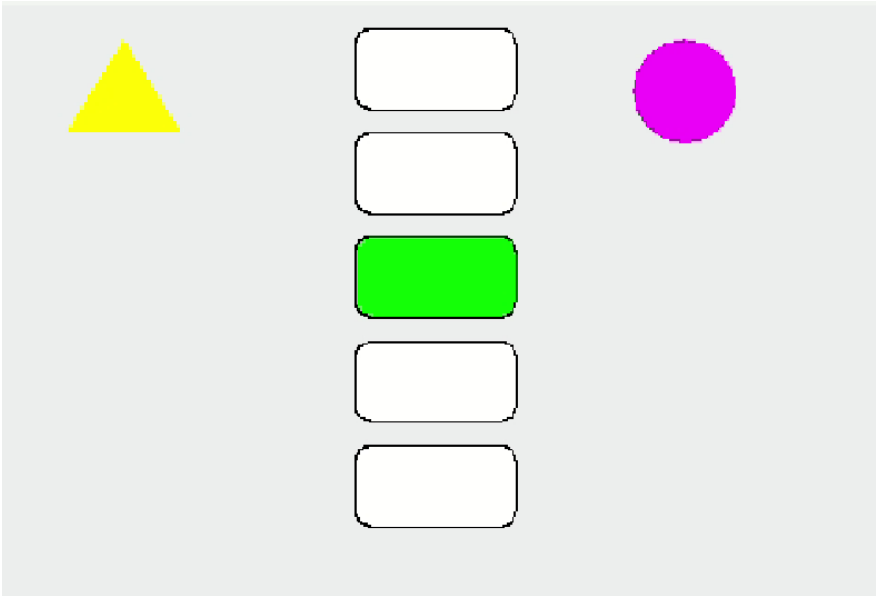
Primera inserción del productor:

El productor adquiere los semáforos huecos y mutex para entrar en la sección crítica (triángulo en azul) y deposita el primer elemento en la posición inicial del buffer (primera celda en verde), mientras el consumidor espera fuera (en amarillo).



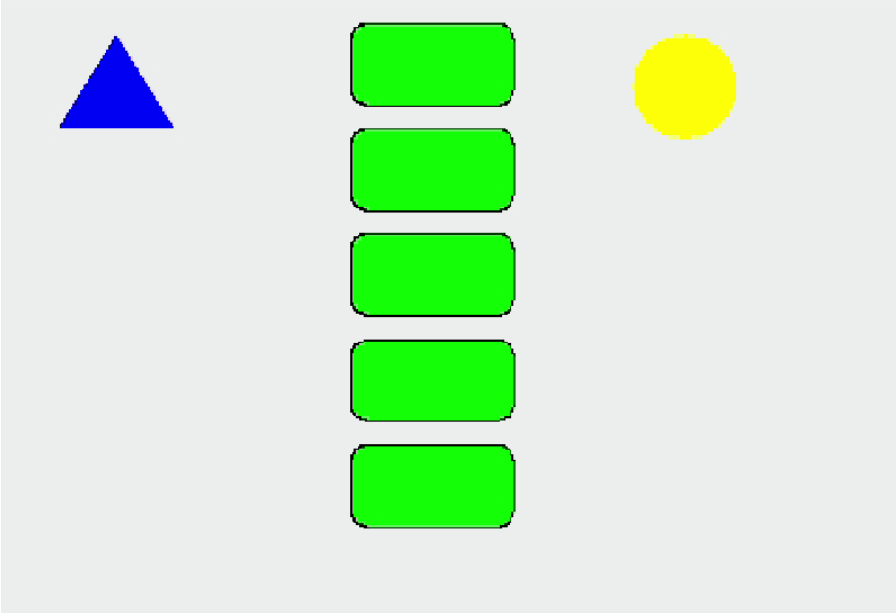
Consumo y rotación circular:

El consumidor entra en la sección crítica (círculo en magenta) y extrae el segundo elemento devolviendo su celda a blanco, quedando ocupada en verde la tercera posición mientras el productor permanece fuera en amarillo. Se aprecia cómo ambos índices avanzan de forma circular por la estructura.



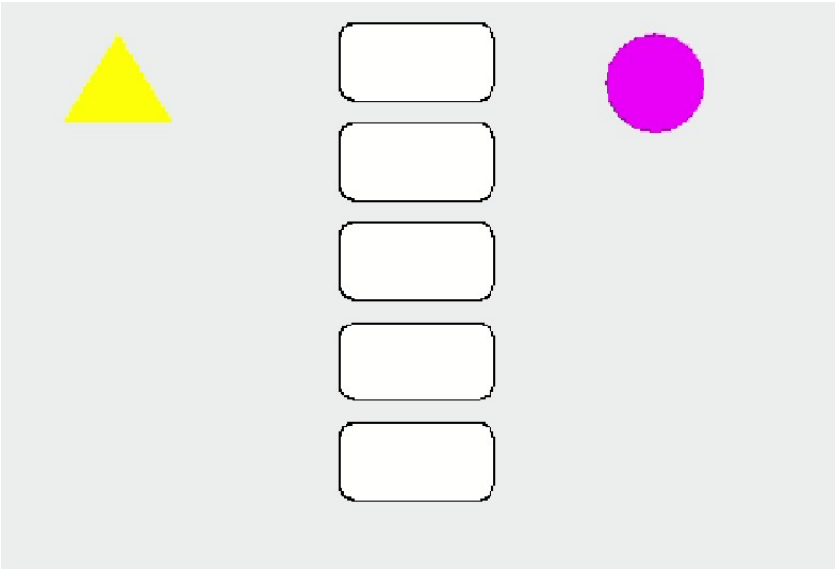
Buffer completamente lleno — capacidad máxima 5/5:

El productor termina de insertar el quinto elemento simultáneo (triángulo en azul y las 5 celdas del buffer en verde). A partir de este instante el semáforo “huecos” vale 0, bloqueando al productor hasta que el consumidor extraiga un elemento y libere espacio.



Última extracción y vaciado final:

El consumidor entra por última vez en la sección crítica (círculo en magenta) tras las 15 iteraciones, vaciando la última celda ocupada para dejar el buffer completamente limpio (las 5 celdas en blanco) antes de finalizar ambos procesos.



2.2. Identificación y Justificación de Tareas

Para resolver el problema del Productor-Consumidor con buffer limitado sin caer en condiciones de carrera ni desperdiciar procesador, la aplicación se organiza en cuatro bloques funcionales que separan la preparación del entorno, el trabajo individual de cada proceso y la sincronización sobre la memoria compartida.

2.2.1. Inicialización del Buffer y de los Mecanismos de Sincronización

Antes de que los procesos empiecen a competir por los recursos, es necesario dejar preparado el espacio común y los semáforos que van a arbitrar su acceso. Se reserva un buffer circular de tamaño fijo (5 posiciones) para acotar la memoria utilizada y se inicializan tres semáforos con roles muy claros:

- **Un semáforo binario de exclusión mutua (mutex):** Su única misión es proteger la integridad de la memoria compartida. Garantiza que, en cualquier instante, solo un proceso pueda manipular el buffer o los punteros de lectura/escritura.
- **Dos semáforos contadores de capacidad (huecos y llenos):** Llevan la cuenta de los recursos disponibles en cada extremo. Gracias a ellos, los procesos saben si pueden avanzar o si deben quedarse suspendidos sin gastar ciclos de CPU haciendo espera activa.

De forma paralela, se prepara la representación gráfica inicial en jBACI para poder observar en tiempo real qué ocurre con cada celda y con cada proceso durante la ejecución.

2.2.2. Trabajo Individual Fuera de la Sección Crítica

Una decisión clave de diseño es mantener la generación de datos (producción) y su posterior procesamiento (consumo) fuera de la sección crítica. Como fabricar o procesar un elemento es una tarea privada de cada proceso que no toca la memoria compartida, no tiene sentido retener el semáforo de exclusión mutua mientras se realizan estas operaciones. De este modo, permitimos una concurrencia real: mientras el consumidor está ocupado procesando el elemento que acaba de sacar, el productor puede seguir fabricando el siguiente dato en paralelo sin quedarse bloqueado innecesariamente.

2.2.3. Acceso Exclusivo y Gestión Circular del Buffer

Cuando un proceso necesita interactuar realmente con el buffer compartido (ya sea para depositar un elemento nuevo o para retirar el más antiguo), entra en su sección crítica protegido por el semáforo de exclusión mutua. Para aprovechar las 5 posiciones de forma continua sin tener que desplazar datos en memoria, el buffer se gestiona de manera circular mediante aritmética modular: cuando cualquiera de los dos índices llega al final del array, vuelve automáticamente a la primera posición. Dentro de este mismo bloque exclusivo se actualiza el estado visual en pantalla (colores de las celdas y de los procesos) para garantizar que la interfaz gráfica siempre refleje el estado real del sistema sin interferencias entre hilos.

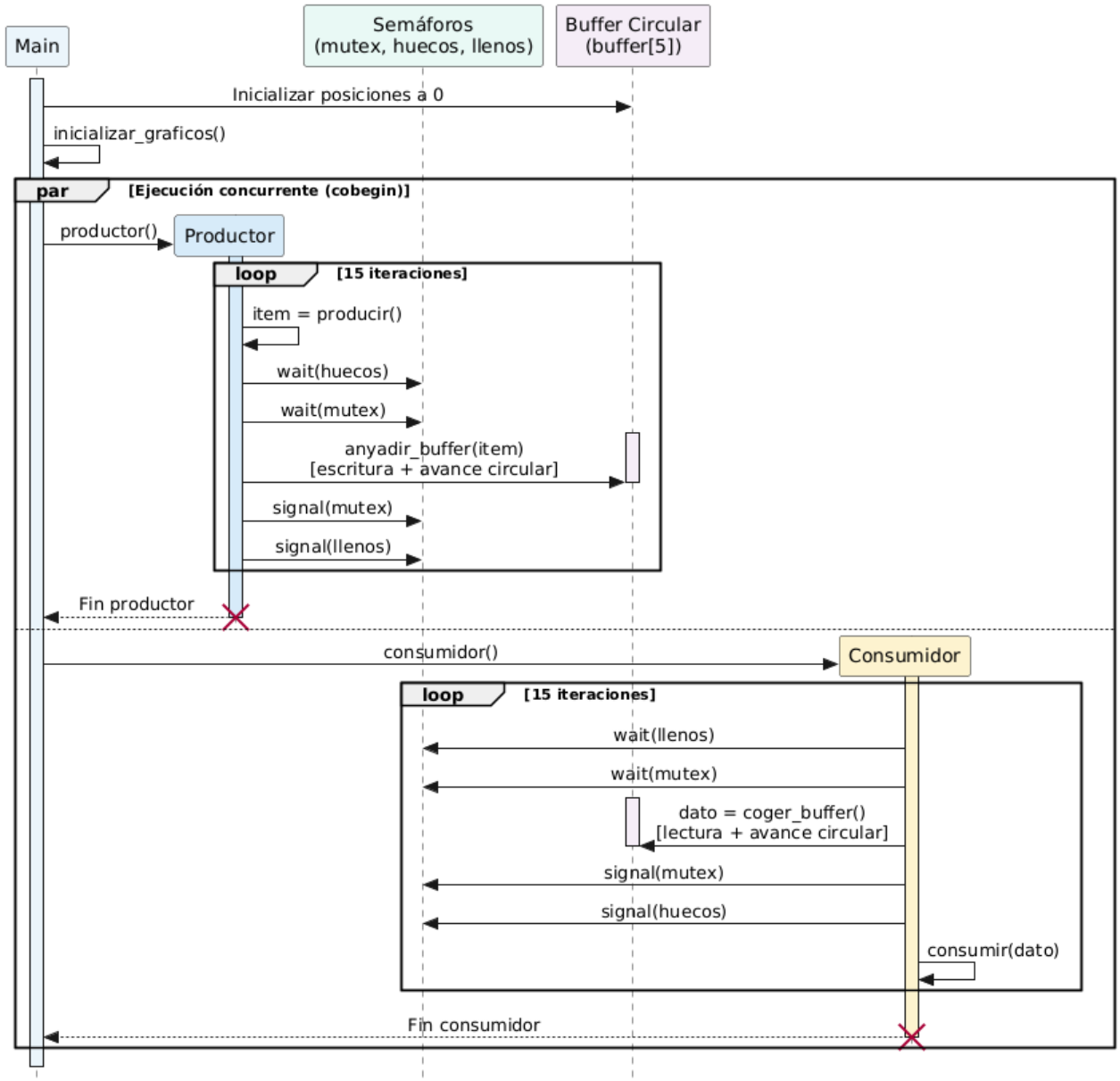
2.2.4. Coordinación entre Procesos y Orden de Bloqueo

Para evitar interbloqueos, ambos procesos siguen un protocolo estricto al entrar y salir de la sección crítica.

Comprobación de disponibilidad antes de pedir exclusión mutua: Tanto el productor como el consumidor verifican primero si hay espacio libre (huecos) o elementos disponibles (llenos) antes de pedir el mutex. Si lo hicieran al revés, un proceso podría entrar en la sección crítica con el buffer lleno o vacío, quedarse bloqueado esperando y dejar la puerta cerrada con llave para el otro proceso, paralizando el sistema para siempre.

Liberación cruzada al terminar: Al salir de la sección crítica, cada proceso suelta primero el mutex para liberar el acceso al buffer y, justo después, incrementa el semáforo contador contrario (llenos o huecos) para despertar al otro proceso en caso de que estuviera dormido esperando.

2.3. Diagrama de Secuencia



2.4. Análisis Detallado del Código Fuente

```
const int TAMANYO_BUFFER = 5;

const
    int RED = 1; int BLACK = 2; int BLUE = 3; int YELLOW = 4; int GREEN = 5;
    int MAGENTA = 6; int WHITE = 7;
const
    int CIRCLE = 1; int LINE = 2; int RECTANGLE = 3; int TRIANGLE = 4; int RNDRECT = 6;

const
    int ID_PROD = 20; int ID_CONS = 30;

const int TIEMPO_PROD = 15;
const int TIEMPO_CONS = 15;
const int TIEMPO_ESPERA = 10;

semaphore mutex = 1;
semaphore huecos = TAMANYO_BUFFER;
semaphore llenos = 0;

int buffer[TAMANYO_BUFFER];

int contProductor = 0;
int contConsumidor = 0;
int dato = 1;

void pausa(int espera)
{
    int k;
    for (k = 0; k < espera; k++)
    {
        // mini pausa para visualizar los estados
    }
}

void inicializar_graficos()
{
    int i;
    for(i = 0; i < TAMANYO_BUFFER; i++)
    {
        create(i + 1, RNDRECT, WHITE, 160, 40 + (i * 45), 70, 35);
    }

    create(ID_PROD, TRIANGLE, YELLOW, 60, 45, 50, 40);
    create(ID_CONS, CIRCLE, YELLOW, 280, 45, 45, 45);
}
```

Recursos globales, control visual y preparación del escenario (pausa e inicializar_graficos): En esta primera parte se declaran los recursos compartidos por todo el sistema: el array circular de 5 posiciones, los índices de lectura y escritura, y los tres semáforos que gobiernan la concurrencia (mutex a 1 para exclusión mutua, huecos a 5 para controlar el espacio libre y llenos a 0 para contabilizar los datos disponibles). Como el entorno de simulación jBACI ejecuta las instrucciones concurrentes demasiado rápido para poder seguirlas a simple vista, pausa() introduce un retardo controlado entre transiciones. Por su parte, inicializar_graficos() construye la interfaz visual antes de arrancar los procesos, dibujando las 5 celdas del buffer en blanco (libres) y situando al productor (triángulo) y al consumidor (círculo) en amarillo en estado de reposo.

```c
int producir()
{
    int nuevo_dato;
    pausa(TIEMPO_PROD);
    nuevo_dato = dato;
    dato = dato + 1;
    return nuevo_dato;
}

void consumir(int dato_sacado)
{
    pausa(TIEMPO_CONS);
}
```

Actividad individual fuera de la sección crítica (producir y consumir): Estas dos funciones representan el trabajo propio que realiza cada proceso de manera independiente sin tocar el buffer compartido. producir() simula el tiempo de generación de un nuevo elemento (TIEMPO_PROD = 15) y devuelve el siguiente valor de la secuencia, mientras que consumir() simula el tiempo que tarda el receptor en procesar el dato ya extraído (TIEMPO_CONS = 15). Mantener ambas tareas fuera de las regiones protegidas por mutex es esencial para lograr concurrencia real, ya que ninguno de los dos procesos bloquea el acceso al buffer mientras está ocupado fabricando o procesando información.

```c
void anyadir_buffer(int item)
{
    buffer[contProductor] = item;

    changecolor(contProductor + 1, GREEN);
    changecolor(ID_PROD, BLUE);
    changecolor(ID_CONS, YELLOW);

    pausa(TIEMPO_ESPERA);

    contProductor = (contProductor + 1) % TAMANYO_BUFFER;
    changecolor(ID_PROD, YELLOW);
}

int coger_buffer()
{
    int dato_sacado;
    dato_sacado = buffer[contConsumidor];

    changecolor(contConsumidor + 1, WHITE);
    changecolor(ID_PROD, YELLOW);
    changecolor(ID_CONS, MAGENTA);

    pausa(TIEMPO_ESPERA);

    contConsumidor = (contConsumidor + 1) % TAMANYO_BUFFER;
    changecolor(ID_CONS, YELLOW);

    return dato_sacado;
}
```

Manipulación en exclusión mutua y rotación circular (anyadir_buffer y coger_buffer): Aquí se encapsulan exactamente las operaciones que forman la sección crítica sobre la memoria compartida:

anyadir_buffer() deposita el nuevo elemento en la posición libre que marca el productor, actualiza el estado visual en pantalla (iluminando la celda en verde y el triángulo del productor en azul durante TIEMPO_ESPERA = 10) y hace avanzar el índice de forma circular mediante contProductor = (contProductor + 1) % TAMANYO_BUFFER para reutilizar las 5 posiciones continuamente.

coger_buffer() realiza la operación simétrica: extrae el elemento más antiguo señalado por el consumidor, devuelve su celda a color blanco, ilumina el círculo del consumidor en magenta durante la breve pausa visual y avanza su propio índice circular antes de devolver el dato extraído.

```
void productor()
{
    int i;
    int item;
    for (i = 0; i < 15; i++)
    {
        item = producir();

        wait(huecos);
        wait(mutex);
        anyadir_buffer(item);
        signal(mutex);
        signal(llenos);
    }
}

void consumidor()
{
    int i;
    int dato_sacado;
    for (i = 0; i < 15; i++)
    {
        wait(llenos);
        wait(mutex);
        dato_sacado = coger_buffer();
        signal(mutex);
        signal(huecos);

        consumir(dato_sacado);
    }
}
void main()
{
    int i;

    for(i = 0; i < TAMANYO_BUFFER; i++)
    {
        buffer[i] = 0;
    }

    inicializar_graficos();

    cobegin
    {
        productor();
        consumidor();
    }
}
```

Protocolo de sincronización concurrente y arranque (productor, consumidor y main): Al haber aislado las tareas auxiliares en las funciones anteriores, tanto productor() como consumidor() reflejan de forma limpia el esquema clásico de sincronización a lo largo de sus 15 iteraciones:

El productor fabrica un elemento, espera a que exista algún hueco disponible (wait(huecos)) y pide exclusión mutua (wait(mutex)) antes de llamar a anyadir_buffer(). Al salir, libera primero el acceso al buffer (signal(mutex)) y después incrementa signal(llenos) para avisar de que hay un nuevo dato listo.

El consumidor sigue el orden inverso: espera a que haya datos en el buffer (wait(llenos)), entra en exclusión mutua (wait(mutex)) para ejecutar coger_buffer(), libera signal(mutex) y avisa de que ha quedado una celda libre (signal(huecos)) antes de ponerse a consumir el dato.

Por último, main() limpia las posiciones iniciales del array, llama a inicializar_graficos() y lanza ambos procesos de manera concurrente dentro de un bloque cobegin, quedando a la espera de que ambos completen sus 15 ciclos para finalizar la ejecución.